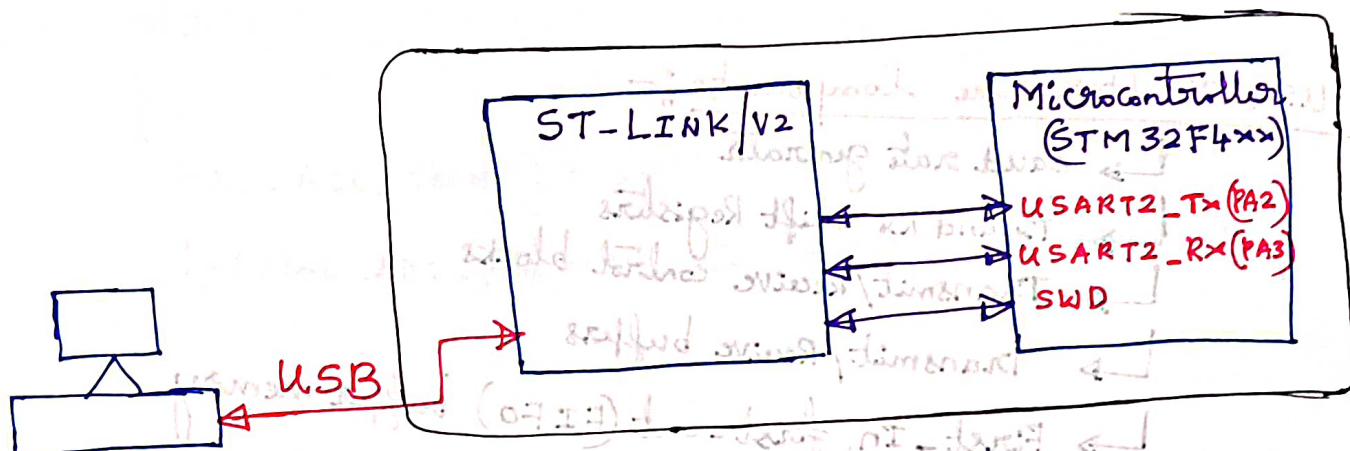


* ST-LINK/V2 USART2 - USB port connection:



↳ The USART2 port is connected to the ST-link programming and debugging tool, which is connected to a USB connector.

↳ The ST-LINK/V2 is an in-circuit debugger and programmer for the STM32 microcontroller families.

↳ SWD: Serial wire debugging Interface

* The three distinct functions of ST-LINK/V2:

1) The ~~downloading~~ programming downloading of the target device

2) The debugging using Serial Wire debug (SWD). The SWD is an alternative to the JTAG standard.

3) Use as a virtual COM port.

3) Interrupt and status Register: (USART_ISR)

↳ USART Interrupt and status Register

↳ This Register contains the flags which reflects the state of sending and receiving data including the availability of the newly received data, the existence of errors in receiving data, the sending unit is ready for new data, Interrupt flag, etc.

* Different clock sources to drive the System clock (SYSCLK):

1. HSI (High speed Internal) RC oscillator clock

2. HSE (High speed External) oscillator clock using a crystal
Value of 4 MHz to 26 MHz

3. PLL clock.

* The SYSCLK (system clock) is the clock that runs the peripherals such as USART. The SYSCLK itself can come from any of the above sources.

↳ The HSI clock signal is generated from an Internal 16 MHz RC oscillator and can be used directly as a system clock or used as a PLL input.

* The HSI oscillator is the default clock source for SYSCLK and is turned on upon reset.

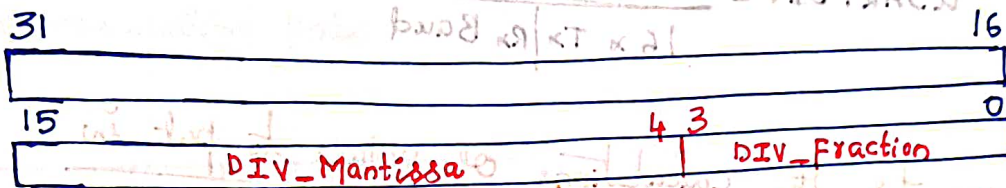
* Baud Register :

↳ Baud Rate Register (USART_BRR)

↳ To generate proper Baud rate, we have to configure this register

↳ This Register has Mantissa and fraction part

↳ The value of the USART_BRR register is called USARTDIV



↳ Bits 31:16 → Reserved, must be kept at reset value

↳ Bits 15:4 → DIV_Mantissa [11:0]: Mantissa of USARTDIV

↳ These 12 bits define the mantissa of the USART divider (USARTDIV)

↳ Bits 3:0 → DIV_Fraction [3:0]: Fraction of USARTDIV

↳ These 4 bits define the fraction of the USART divider (USARTDIV).

* Even though the Baud Register (USART_BRR) is a 32-bit Register only the lower 16-bits are used.

↳ For BRR register, we have the choice of oversampling of 8 or 16.

↳ Formula to get USARTDIV value

$$Tx/Rx \text{ baud} = \frac{f_{ck}}{8 \times (2 - \text{OVER8}) \times \text{USARTDIV}}$$

for oversampling by 16 (OVER8 = 0, default), we get

USART_BRR = USARTDIV as follows

$$Tx/Rx \text{ Baud} = \frac{f_{ck}}{8 \times (2 - 0) \times \text{USARTDIV}}$$

$$Tx/Rx \text{ Baud} = \frac{f_{ck}}{16 \times \text{USARTDIV}}$$

$$\text{USARTDIV} = \frac{f_{ck}}{16 \times Tx/Rx \text{ Baud}}$$

* Examples for the calculations of values to put in USART_BRR Baud Register

- 1) Assume clock frequency 16 MHz is given to USART Baud rate generator with oversampling rate of 16. Find the values for the USART_BRR register for baud rates of a) 9600 b) 19,200 c) 57,600 and d) 115,200

Ans : a) We have Equation,

$$\text{USARTDIV} = \frac{f_{ck}}{16 \times \text{Baud}}$$

$$= \frac{16 \times 10^6}{16 \times 9600}$$

$$= \frac{1000000}{9600}$$

$$= 104.166$$

$$\text{DIV_Mantissa (12 bits upper)} = 104 = 0 \times 68$$

$$\text{DIV_Fraction (4 bits Lower)} = 0.166 \times 16 = 2.656 = 3$$

$$\therefore \text{BRR} = 0 \times 0683$$

Baudrate	BRR(index)	BRR(in hex)
9,600	104.166	0x0683
19,200	52.06	0x0341
57,600	17.36	0x0116
115,200	8.68	0x008B

NOTE:

→ STM32F4xx micro controller provides 4 USART and 2 UART communication ports

U(S)ARTx	Tx	Rx	Tx	Rx	Tx	Rx	APB
USART1	PA9	PA10	PB6	PB7			2
USART2	PA2	PA3	PD5	PD6			1
USART3	PB10	PB11	PC10	PC11	PD8	PD9	1
UART4	PA0	PA1	PC10	PC11			1
UART5	PC12	PD2					1
USART6	PC6	PC7	PQ14	PQ9			2

→ The above Table shows the GPIO pins corresponding to each communication port Tx/Rx pins and the bus to which USART module is connected and through which a clock is provided to each port.

→ USART1 and USART6 are connected to APB2 bus and APB2 bus can run at maximum of 84 MHz. Therefore, maximum operating frequency for USART1 and USART6 is 84 MHz.

* Timers: programming with timers:

* Timers: programming with timers: "blue"

Develop a 'C' code and necessary settings to toggle on board LED with a time period of 500ms using Timer-2 and Assume clock frequency = 16 MHz.

program:

```
#include "main.h"
```

```
/* private variable */
```

```
TIM_HandleTypeDef htim2;
```

```
int main(void)
```

```
int main(void)
{
    /* Initialize all configured peripherals */
}
```

```
mx_gpio_init();
```

`MX_TIM2_Init();`

```
HAL_TIM_Base_Start_IT(&htim2);
```

```
while(1)
```

 $\{$

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef * htim)
```

```
HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_15);
```

3

* STM32CubeMX configuration:

* STM32CubeMx Configuration:

1) System core $\rightarrow$ Rcc $\rightarrow$ High speed clock (HSE) $\rightarrow$ crystal ceramic resonator

$\rightarrow$ Sys $\rightarrow$ Debug $\rightarrow$ Serial wire

sys → Debug → Serial wire

2> Timers $\rightarrow$ TIM2 (Timer 2) $\rightarrow$ clock source Internal clock
parameter settings:

parameter settings:

prescaler: 1000

Counter Mode: up

Counter Mode: up
Counter period (Auto Reload): 8000

auto reload preload: Enable